

Unidad 3 - Sesión 2: Acceso Programático a LLMs

Objetivos de la Sesión

Al finalizar esta sesión, el estudiante será capaz de:

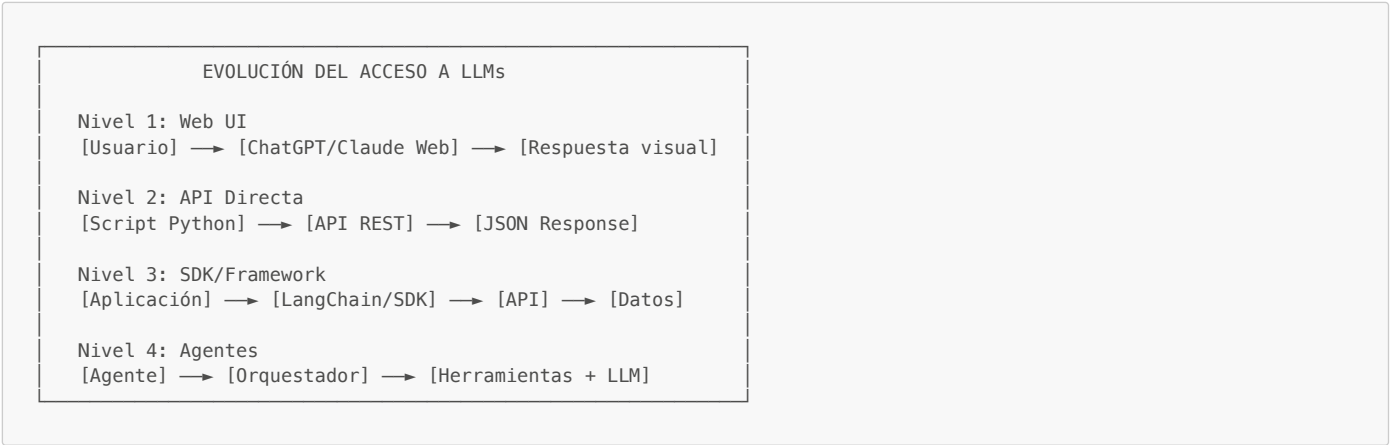
- Comprender la arquitectura de las APIs REST de los principales proveedores de LLMs
- Integrar las APIs de OpenAI, Google Gemini y Anthropic Claude en aplicaciones Python
- Implementar patrones de producción: manejo de errores, reintentos, streaming y gestión de costes
- Desarrollar casos prácticos: chatbots con memoria, análisis de sentimiento, extracción estructurada
- Comprender los fundamentos de Function Calling y generación de embeddings
- Introducir LangChain como framework de orquestación para aplicaciones con LLMs

Bloque 1: Fundamentos de APIs de LLMs (40 minutos)

1.1 De Interfaces Gráficas a Código

Hasta ahora hemos interactuado con los LLMs a través de interfaces gráficas (ChatGPT, Claude.ai, Gemini). El acceso programático abre un nuevo mundo de posibilidades.

Aspecto	Interfaz Gráfica (Web)	Acceso Programático (API)
Interacción	Manual, un prompt a la vez	Automatizada, miles de llamadas
Datos	Copiar/pegar resultados	Integración directa con sistemas
Configuración	Opciones limitadas de la UI	Control total de parámetros
Escalabilidad	No escalable	Escalable a millones de peticiones
Reproducibilidad	Difícil de reproducir	100% reproducible con mismos parámetros
Integración	Aislado del resto del sistema	Parte de pipelines y aplicaciones



1.2 Conceptos de APIs REST para LLMs

Una **API REST** (Representational State Transfer) permite la comunicación entre sistemas mediante peticiones HTTP. Para los LLMs, las llamadas siguen un patrón común.

Anatomía de una Petición

```
POST https://api.openai.com/v1/chat/completions

Headers:
Content-Type: application/json
Authorization: Bearer sk-xxxxxxxxxxxxxxxxxxxxx

Body (JSON):
{
  "model": "gpt-4o",
  "messages": [
    {"role": "system", "content": "Eres un asistente útil."},
    {"role": "user", "content": "¿Qué es una API?"}
  ],
  "temperature": 0.7,
  "max_tokens": 500
}
```

Componentes Clave

PETICIÓN HTTP	
MÉTODO:	POST (siempre para generación)
URL:	Endpoint del proveedor
HEADERS:	Authorization + Content-Type
BODY:	Modelo + Mensajes + Parámetros
RESPUESTA HTTP	
STATUS:	200 OK / 4xx Error cliente / 5xx Error srv
BODY:	JSON con respuesta, uso de tokens, metadata

Códigos de Estado HTTP Relevantes

Código	Significado	Acción Recomendada
200	Éxito	Procesar la respuesta
400	Petición mal formada	Revisar parámetros y formato del body
401	No autorizado	Verificar API Key
403	Prohibido	Revisar permisos de la cuenta
429	Rate limit excedido	Esperar y reintentar con backoff
500	Error interno del servidor	Reintentar después de una pausa
503	Servicio no disponible	El proveedor tiene problemas, reintentar

1.3 Estructura de Costos y Optimización

Los proveedores de LLMs cobran por **tokens** procesados, diferenciando entre tokens de entrada (input) y tokens de salida (output).

Precios por Proveedor (por 1M de tokens)

Modelo	Input (1M tokens)	Output (1M tokens)	Contexto Máx
GPT-4o	\$2.50	\$10.00	128K
GPT-4o-mini	\$0.15	\$0.60	128K
GPT-3.5-turbo	\$0.50	\$1.50	16K
Claude 3.5 Sonnet	\$3.00	\$15.00	200K
Claude 3 Haiku	\$0.25	\$1.25	200K
Gemini 1.5 Pro	\$1.25	\$5.00	1M
Gemini 1.5 Flash	\$0.075	\$0.30	1M

Nota: Los precios se actualizan frecuentemente. Consultar siempre la documentación oficial del proveedor.

Ejemplo de Cálculo de Costos

```
Escenario: Análisis de sentimiento de 10,000 reseñas

Prompt del sistema:      50 tokens   (fijo)
Cada reseña (promedio):  200 tokens  (variable)
Respuesta (promedio):    50 tokens   (variable)

Total input:  (50 + 200) × 10,000 = 2,500,000 tokens
Total output: 50 × 10,000         =   500,000 tokens

Costo con GPT-4o:
Input:  2.5M × $2.50/1M  = $6.25
Output: 0.5M × $10.00/1M = $5.00
Total:                                $11.25

Costo con GPT-4o-mini:
Input:  2.5M × $0.15/1M  = $0.375
Output: 0.5M × $0.60/1M  = $0.30
Total:                                $0.675
```

5 Estrategias de Optimización de Costos

- 1. **Seleccionar el modelo adecuado:** Usar modelos más pequeños para tareas simples (GPT-4o-mini en lugar de GPT-4o)
- 2. **Minimizar tokens de entrada:** System prompts concisos, eliminar contexto innecesario
- 3. **Limitar tokens de salida:** Usar `max_tokens` para evitar respuestas excesivamente largas
- 4. **Implementar caché:** Almacenar respuestas para prompts repetidos
- 5. **Procesamiento por lotes:** Agrupar múltiples tareas en una sola llamada cuando sea posible
- 6. **Usar API Gateways con modelos gratuitos:** Servicios como [OpenRouter](#) ofrecen acceso a modelos gratuitos de múltiples proveedores con coste cero (ver sección 3.4)

1.4 Rate Limiting y Manejo de Errores

Los proveedores imponen límites para proteger sus servicios y garantizar acceso equitativo.

Tipos de Límites

Tipo	Significado	Ejemplo
RPM	Peticiones por minuto	500 RPM
TPM	Tokens por minuto	200,000 TPM
RPD	Peticiones por día	10,000 RPD

Implementación de Retry con Exponential Backoff

```
import time
import random

def llamar_con_reintentos(funcion, max_reintentos=5, base_delay=1):
    """
    Reintenta una llamada a la API con exponential backoff y jitter.

    Args:
        funcion: Función que realiza la llamada a la API
        max_reintentos: Número máximo de reintentos
        base_delay: Tiempo base de espera en segundos

    Returns:
        Resultado de la función si tiene éxito

    Raises:
        Exception: Si se agotan todos los reintentos
    """
    for intento in range(max_reintentos):
        try:
            return funcion()
        except Exception as e:
            if intento == max_reintentos - 1:
                raise # Último intento, propagar el error

            # Exponential backoff con jitter
            delay = base_delay * (2 ** intento) + random.uniform(0, 1)
            print(f"Intento {intento + 1} fallido: {e}")
            print(f"Reintentando en {delay:.1f} segundos...")
            time.sleep(delay)
```

Patrón de Exponential Backoff:

Intento 1: falla → espera ~1s
 Intento 2: falla → espera ~2s
 Intento 3: falla → espera ~4s
 Intento 4: falla → espera ~8s
 Intento 5: falla → error final

El "jitter" (ruido aleatorio) evita que múltiples clientes reintenten exactamente al mismo tiempo (thundering herd problem).

1.5 Gestión Segura de API Keys

Las API Keys son credenciales secretas que autorizan el acceso a los servicios. Su manejo seguro es **crítico**.

REGLA DE ORO: NUNCA incluyas API Keys directamente en el código fuente.

```
# ❌ NUNCA hacer esto
client = OpenAI(api_key="sk-abc123456789...")

# ❌ NUNCA subir keys a repositorios
# Ni siquiera en repositorios privados
```

Opción 1: Variables de Entorno

```
# En la terminal (Linux/Mac)
export OPENAI_API_KEY="sk-tu-clave-aquí"
export ANTHROPIC_API_KEY="sk-ant-tu-clave-aquí"
export GOOGLE_API_KEY="tu-clave-aquí"
```

```
import os

# El SDK de OpenAI busca automáticamente la variable OPENAI_API_KEY
from openai import OpenAI
client = OpenAI() # Lee OPENAI_API_KEY del entorno

# Para otros proveedores
api_key = os.getenv("ANTHROPIC_API_KEY")
```

```
if not api_key:
    raise ValueError("ANTHROPIC_API_KEY no configurada")
```

Opción 2: Archivo .env con python-dotenv

```
# Archivo .env (en la raíz del proyecto)
OPENAI_API_KEY=sk-tu-clave-aquí
ANTHROPIC_API_KEY=sk-ant-tu-clave-aquí
GOOGLE_API_KEY=tu-clave-aquí
OPENROUTER_API_KEY=sk-or-tu-clave-aquí # Alternativa gratuita (ver sección 3.4)
```

```
from dotenv import load_dotenv
import os

# Cargar variables del archivo .env
load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")
```

```
# IMPORTANTE: Añadir .env al .gitignore
echo ".env" >> .gitignore
```

Opción 3: Google Drive (para Google Colab)

```
# En Google Colab
from google.colab import drive, userdata

# Opción A: Usar Secrets de Colab (recomendado)
api_key = userdata.get("OPENAI_API_KEY")

# Opción B: Archivo en Google Drive
drive.mount('/content/drive')
with open('/content/drive/MyDrive/keys/openai_key.txt', 'r') as f:
    api_key = f.read().strip()
```

Bloque 2: API de OpenAI (50 minutos)

2.1 Instalación del SDK y Configuración

```
# Instalación
!pip install openai python-dotenv

# Importación y configuración
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

# El cliente busca automáticamente OPENAI_API_KEY en las variables de entorno
client = OpenAI()
```

Alternativa: Usar OpenRouter como Proxy

Si no dispones de una API Key de OpenAI, puedes usar **OpenRouter** como proxy. OpenRouter utiliza la misma librería **openai**, solo cambia la configuración del cliente:

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()

# Cliente configurado para OpenRouter (en lugar de OpenAI directo)
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
```

```

    api_key=os.getenv("OPENROUTER_API_KEY"),
)

# Usar un modelo gratuito en lugar de gpt-4o-mini
response = client.chat.completions.create(
    model="google/gemini-2.0-flash-exp:free", # Modelo gratuito
    messages=[{"role": "user", "content": "Hola, ¿qué es una API?"}]
)
print(response.choices[0].message.content)

```

Nota: El resto de los ejemplos de esta sesión usan `client = OpenAI()` con modelos de OpenAI. Si usas OpenRouter, simplemente sustituye la inicialización del cliente y el nombre del modelo como se muestra arriba.

2.2 Primera Llamada: Chat Completions

```

from openai import OpenAI

client = OpenAI()

# Primera llamada básica
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "user", "content": "¿Qué es una API REST? Responde en 2 oraciones."}
    ]
)

# Extraer la respuesta
respuesta = response.choices[0].message.content
print(respuesta)

# Inspeccionar el uso de tokens
print(f"\nTokens de entrada: {response.usage.prompt_tokens}")
print(f"Tokens de salida: {response.usage.completion_tokens}")
print(f"Tokens totales: {response.usage.total_tokens}")

```

Estructura de la Respuesta

```

# El objeto response contiene:
response.id          # ID único de la respuesta
response.model       # Modelo utilizado
response.choices[0].message.role # "assistant"
response.choices[0].message.content # Texto de la respuesta
response.choices[0].finish_reason # "stop", "length", etc.
response.usage.prompt_tokens # Tokens del prompt
response.usage.completion_tokens # Tokens generados
response.usage.total_tokens # Total de tokens

```

2.3 Estructura de Mensajes (system, user, assistant)

La API de Chat Completions utiliza un sistema de mensajes con tres roles fundamentales.

Rol **system**: Configura el Comportamiento

```

messages = [
    {
        "role": "system",
        "content": "Eres un experto en Python. Respondes siempre con "
                    "código comentado y explicaciones breves."
    }
]

```

El mensaje de sistema establece el comportamiento base del modelo durante toda la conversación.

Rol **user**: Mensajes del Usuario

```
messages.append({
    "role": "user",
    "content": "¿Cómo ordeno una lista de diccionarios por un campo?"
})
```

Rol **assistant**: Mantener el Contexto

```
messages.append({
    "role": "assistant",
    "content": "Puedes usar sorted() con una función lambda:\n\n"
               "``python\nusuarios = [{'nombre': 'Ana', 'edad': 30}, ...]\n"
               "ordenados = sorted(usuarios, key=lambda x: x['edad'])\n``"
```

Ejemplo Completo: Conversación Multi-turno

```
from openai import OpenAI

client = OpenAI()

# Historial de la conversación
mensajes = [
    {
        "role": "system",
        "content": "Eres un tutor de Python amigable. Respondes de forma "
                  "concisa y con ejemplos prácticos."
    },
    {
        "role": "user",
        "content": "¿Qué es una list comprehension?"
    },
    {
        "role": "assistant",
        "content": "Una list comprehension es una forma concisa de crear "
                  "listas en Python.\n\n"
                  "``python\n"
                  "# Forma tradicional\n"
                  "cuadrados = []\n"
                  "for x in range(5):\n"
                  "    cuadrados.append(x**2)\n\n"
                  "# Con list comprehension\n"
                  "cuadrados = [x**2 for x in range(5)]\n"
                  "# Resultado: [0, 1, 4, 9, 16]\n"
                  "```\n\n"
    },
    {
        "role": "user",
        "content": "¿Puedo añadir condiciones?"
    }
]

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=mensajes
)

# El modelo tiene contexto de toda la conversación anterior
print(response.choices[0].message.content)
```

FLUJO DE MENSAJES EN LA API:

<pre>messages = [{system: "Eres un tutor de Python..."}, {user: "¿Qué es una list comprehension?"}, {assistant: "Una list comprehension es..."}, {user: "¿Puedo añadir condiciones?"},]</pre>	← Comportamiento base ← Pregunta 1 ← Respuesta 1 ← Pregunta 2 (actual)
→ El modelo tiene TODO el historial → Entiende que "condiciones" se refiere a	

```
list comprehensions
```

2.4 Parámetros de la API

```
response = client.chat.completions.create(
    # --- Obligatorios ---
    model="gpt-4o",           # Modelo a utilizar
    messages=[...],          # Lista de mensajes

    # --- Parámetros de Generación ---
    temperature=0.7,          # Creatividad: 0.0 (determinista) a 2.0 (muy aleatorio)
    max_tokens=1000,          # Máximo de tokens en la respuesta
    top_p=0.9,                # Nucleus sampling: probabilidad acumulada
    frequency_penalty=0.0,     # Penaliza repetición de tokens (-2.0 a 2.0)
    presence_penalty=0.0,      # Penaliza repetición de temas (-2.0 a 2.0)

    # --- Control de Salida ---
    stop=["\n\n", "FIN"],     # Tokens donde detener la generación
    n=1,                      # Número de respuestas a generar

    # --- Formato ---
    response_format={"type": "json_object"}, # Forzar salida JSON (cuando aplique)

    # --- Streaming ---
    stream=False,              # True para respuesta progresiva
)
```

Recomendaciones de Configuración por Caso de Uso

Caso de Uso	temperature	top_p	max_tokens	Notas
Precisión / Datos	0.0 - 0.2	0.9	Variable	Respuestas consistentes
Conversación general	0.5 - 0.7	0.9	500-1000	Balance calidad/variedad
Escritura creativa	0.8 - 1.2	0.95	1000+	Mayor variedad
Determinista (tests)	0.0	1.0	Variable	Misma respuesta siempre
Código / JSON	0.0 - 0.2	0.9	Variable	Priorizar corrección

Nota: No se recomienda modificar `temperature` y `top_p` simultáneamente. Ajustar uno y dejar el otro en su valor por defecto.

2.5 Streaming de Respuestas

El streaming permite recibir la respuesta token a token, mejorando la experiencia del usuario en interfaces interactivas.

```
from openai import OpenAI

client = OpenAI()

# Llamada con streaming
stream = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "user", "content": "Explica qué es Docker en 3 párrafos."}
    ],
    stream=True # Habilitar streaming
)

# Procesar chunks a medida que llegan
respuesta_completa = ""
for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        fragmento = chunk.choices[0].delta.content
        print(fragmento, end="", flush=True)
        respuesta_completa += fragmento

print() # Salto de línea final
```

Streaming en Aplicaciones Web (Flask/FastAPI)

```

from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from openai import OpenAI

app = FastAPI()
client = OpenAI()

def generar_stream(prompt: str):
    """Generador que produce chunks de texto para streaming HTTP."""
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        stream=True
    )
    for chunk in stream:
        if chunk.choices[0].delta.content is not None:
            yield chunk.choices[0].delta.content

@app.get("/chat")
async def chat(prompt: str):
    return StreamingResponse(
        generar_stream(prompt),
        media_type="text/plain"
    )

```

2.6 Manejo de Errores en Producción

```

import time
import random
from openai import OpenAI, RateLimitError, APIConnectionError, APIError

client = OpenAI()

def llamar_api_robusto(mensajes, modelo="gpt-4o-mini", max_reintentos=3):
    """
    Llamada robusta a la API de OpenAI con manejo completo de errores.

    Args:
        mensajes: Lista de mensajes para la conversación
        modelo: Modelo a utilizar
        max_reintentos: Número máximo de reintentos

    Returns:
        Texto de la respuesta del modelo

    Raises:
        Exception: Si se agotan los reintentos o hay un error no recuperable
    """
    for intento in range(max_reintentos):
        try:
            response = client.chat.completions.create(
                model=modelo,
                messages=mensajes,
                timeout=30 # Timeout de 30 segundos
            )
            return response.choices[0].message.content

        except RateLimitError as e:
            # Rate limit: esperar con exponential backoff
            espera = (2 ** intento) + random.uniform(0, 1)
            print(f"Rate limit alcanzado. Esperando {espera:.1f}s... "
                  f"({intento + 1}/{max_reintentos})")
            time.sleep(espera)

        except APIConnectionError as e:
            # Error de conexión: reintentar
            print(f"Error de conexión: {e}. "
                  f"Reintentando ({intento + 1}/{max_reintentos})...")
            time.sleep(2)

        except APIError as e:
            # Error del servidor (5xx): reintentar
            if e.status_code and e.status_code >= 500:
                print(f"Error del servidor ({e.status_code}). "
                      f"Reintentando ({intento + 1}/{max_reintentos})...")
                time.sleep(3)
            else:

```

```
        # Error del cliente (4xx): no reintentar
        raise

    raise Exception(f"Se agotaron los {max_reintentos} reintentos")

# Uso
respuesta = llamar_api_robusto(
    mensajes=[
        {"role": "system", "content": "Eres un asistente útil."},
        {"role": "user", "content": "Hola, ¿cómo estás?"}
    ]
)
print(respuesta)
```

Bloque 3: APIs de Google Gemini y Anthropic Claude (40 minutos)

3.1 Google Gemini - SDK google-generativeai

Instalación y Configuración

```
!pip install google-generativeai

import google.generativeai as genai
import os

# Configurar la API Key
genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))

# Crear el modelo
model = genai.GenerativeModel("gemini-1.5-flash")
```

Primera Llamada

```
# Llamada simple
response = model.generate_content("¿Qué es una API REST? Responde en 2 oraciones.")
print(response.text)
```

Chat con Historial

```
# Iniciar un chat con historial
chat = model.start_chat(history=[])

# Primera pregunta
response1 = chat.send_message("¿Qué es Python?")
print(response1.text)

# Segunda pregunta (mantiene contexto automáticamente)
response2 = chat.send_message("¿Cuáles son sus principales bibliotecas de datos?")
print(response2.text)

# Ver el historial completo
for message in chat.history:
    print(f"{message.role}: {message.parts[0].text[:80]}...")
```

Configuración de Generación y Seguridad

```
from google.generativeai.types import GenerationConfig, HarmCategory, HarmBlockThreshold

# Configuración de generación
generation_config = GenerationConfig(
    temperature=0.7,
    top_p=0.9,
    top_k=40,
    max_output_tokens=1000,
)

# Configuración de seguridad
safety_settings = {
```

```

HarmCategory.HARM_CATEGORY_HARASSMENT: HarmBlockThreshold.BLOCK_MEDIUM_AND_ABOVE,
HarmCategory.HARM_CATEGORY_HATE_SPEECH: HarmBlockThreshold.BLOCK_MEDIUM_AND_ABOVE,
}

# Crear modelo con configuración
model = genai.GenerativeModel(
    "gemini-1.5-flash",
    generation_config=generation_config,
    safety_settings=safety_settings,
    system_instruction="Eres un tutor de programación amigable."
)

```

Capacidades Multimodales (Imágenes)

```

import PIL.Image

# Cargar una imagen
imagen = PIL.Image.open("diagrama.png")

# Enviar imagen + texto al modelo
model = genai.GenerativeModel("gemini-1.5-flash")
response = model.generate_content(
    ["Describe qué muestra esta imagen:", imagen]
)
print(response.text)

```

3.2 Anthropic Claude - SDK anthropic

Instalación y Configuración

```

!pip install anthropic

import anthropic
import os

# Crear el cliente
client = anthropic.Anthropic(
    api_key=os.getenv("ANTHROPIC_API_KEY")
)

```

Primera Llamada

```

message = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1000,
    messages=[
        {"role": "user", "content": "¿Qué es una API REST? Responde en 2 oraciones."}
    ]
)

print(message.content[0].text)

```

Diferencia Clave: System Prompt como Parámetro Separado

A diferencia de OpenAI, donde el system prompt es un mensaje más en la lista, en Claude el **system prompt es un parámetro separado** de la función `create()`.

```

message = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1000,
    system="Eres un experto en Python. Respondes siempre con código "
        "comentado y explicaciones breves.", # ← Parámetro separado
    messages=[
        {"role": "user", "content": "¿Cómo leo un archivo CSV?"}
    ]
)

print(message.content[0].text)

```

Streaming con Claude

```
# Streaming usando context manager
with client.messages.stream(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1000,
    messages=[
        {"role": "user", "content": "Explica qué es Docker en 3 párrafos."}
    ]
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)

print()
```

Conversación Multi-turno

```
# Claude requiere alternar roles user/assistant
mensajes = [
    {"role": "user", "content": "¿Qué es una list comprehension en Python?"},
    {"role": "assistant", "content": "Una list comprehension es una forma "
                                     "concisa de crear listas.."},
    {"role": "user", "content": "¿Puedo añadir condiciones?"}
]

message = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1000,
    system="Eres un tutor de Python.",
    messages=mensajes
)
print(message.content[0].text)
```

3.3 Comparativa de Sintaxis entre las Tres APIs

```
# =====
# === OPENAI ===
# =====
from openai import OpenAI

client_openai = OpenAI()

response = client_openai.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Eres un asistente útil."},
        {"role": "user", "content": "Hola, ¿qué tal?"}
    ],
    temperature=0.7,
    max_tokens=500
)
print(response.choices[0].message.content)

# =====
# === GOOGLE GEMINI ===
# =====
import google.generativeai as genai

genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))

model = genai.GenerativeModel(
    "gemini-1.5-flash",
    system_instruction="Eres un asistente útil."
)
response = model.generate_content("Hola, ¿qué tal?")
print(response.text)

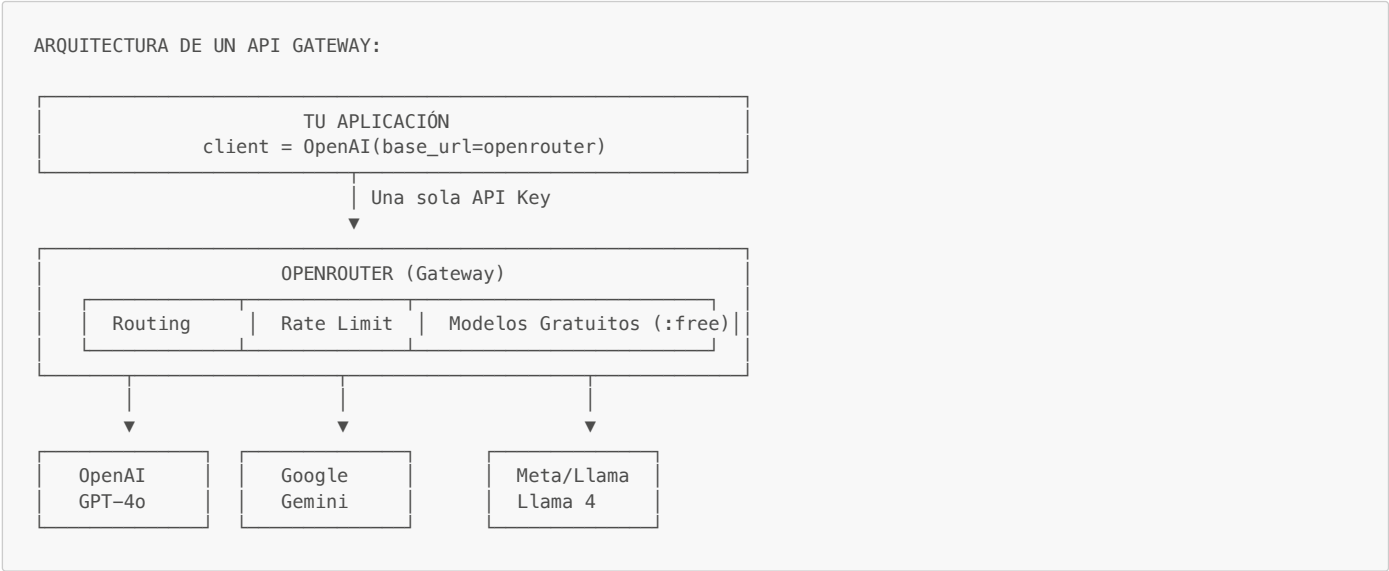
# =====
# === ANTHROPIC CLAUDE ===
# =====
import anthropic

client_claude = anthropic.Anthropic()
```

```
message = client_claude.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=500,
    system="Eres un asistente útil.",
    messages=[
        {"role": "user", "content": "Hola, ¿qué tal?"}
    ]
)
print(message.content[0].text)
```

3.4 OpenRouter: API Gateway Unificado

OpenRouter (openrouter.ai) es un **API Gateway** que actúa como proxy unificado para acceder a modelos de múltiples proveedores (OpenAI, Google, Anthropic, Meta, Mistral y más) a través de una **única API compatible con OpenAI**.



Ventajas de un API Gateway

Ventaja	Descripción
Una sola API Key	Acceso a decenas de proveedores con una credencial
Modelos gratuitos	Modelos con sufijo <code>:free</code> sin coste (ideales para aprendizaje)
Misma librería	Usa <code>openai</code> de Python, solo cambia <code>base_url</code>
Comparar modelos fácilmente	Cambiar de proveedor es cambiar el nombre del modelo
Fallback automático	Si un proveedor falla, puede enrutar a otro

Configuración

```
from openai import OpenAI
import os

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.getenv("OPENROUTER_API_KEY"),
)
```

Modelos Gratuitos Disponibles

Consulta la lista actualizada en: <https://openrouter.ai/models?q=free&order=most-popular>

Modelo	ID en OpenRouter	Proveedor
Gemini 2.0 Flash	<code>google/gemini-2.0-flash-exp:free</code>	Google
DeepSeek R1	<code>deepseek/deepseek-r1-0528:free</code>	DeepSeek
Llama 4 Scout	<code>meta-llama/llama-4-scout:free</code>	Meta
Qwen3 30B	<code>qwen/qwen3-30b-a3b:free</code>	Alibaba

Ejemplo: Comparar Modelos con una Sola API

```
from openai import OpenAI
import os
import time

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.getenv("OPENROUTER_API_KEY"),
)

modelos = {
    "Gemini 2.0 Flash": "google/gemini-2.0-flash-exp:free",
    "Llama 4 Scout": "meta-llama/llama-4-scout:free",
    "Qwen3 30B": "qwen/qwen3-30b-a3b:free",
}

prompt = "¿Qué es una API REST? Responde en 2 oraciones."

for nombre, modelo_id in modelos.items():
    start = time.time()
    response = client.chat.completions.create(
        model=modelo_id,
        messages=[{"role": "user", "content": prompt}],
        temperature=0.7,
    )
    elapsed = time.time() - start

    print(f"=== {nombre} ({elapsed:.2f}s) ===")
    print(response.choices[0].message.content)
    print()
```

Nota: Los modelos gratuitos tienen límites de uso (rate limits) más estrictos que los de pago. Para proyectos de producción se recomienda usar las APIs directas de cada proveedor.

Tabla Comparativa de las Tres APIs

Aspecto	OpenAI	Google Gemini	Anthropic Claude	OpenRouter
Instalación	<code>pip install openai</code>	<code>pip install google-generativeai</code>	<code>pip install anthropic</code>	<code>pip install openai</code>
System prompt	Mensaje con <code>role: "system"</code>	<code>system_instruction</code> en modelo	Parámetro <code>system</code> en <code>create()</code>	Mensaje con <code>role: "system"</code>
Acceso a respuesta	<code>response.choices[0].message.content</code>	<code>response.text</code>	<code>message.content[0].text</code>	<code>response.choices[0].message.content</code>
Contexto máximo	128K tokens	1M tokens	200K tokens	Depende del modelo
Multimodal nativo	Sí (GPT-4o)	Sí (nativo)	Sí (imágenes)	Depende del modelo
Streaming	<code>stream=True</code> en <code>create()</code>	<code>generate_content(stream=True)</code>	<code>client.messages.stream()</code>	<code>stream=True</code> en <code>create()</code>
Variable de entorno	<code>OPENAI_API_KEY</code>	<code>GOOGLE_API_KEY</code>	<code>ANTHROPIC_API_KEY</code>	<code>OPENROUTER_API_KEY</code>
Modelos gratuitos	No	Sí (tier gratuito)	No	Sí (sufijo <code>:free</code>)

Bloque 4: Casos Prácticos (50 minutos)

4.1 Chatbot con Memoria de Conversación

Un chatbot que mantiene el historial de la conversación para proporcionar respuestas contextualizadas.

Código completo: [chatbot.py](#)

Patrón Fundamental

```
from openai import OpenAI

client = OpenAI()

class Chatbot:
    def __init__(self, system_prompt, modelo="gpt-4o-mini", max_mensajes=20):
```

```

self.modelo = modelo
self.max_mensajes = max_mensajes
self.mensajes = [
    {"role": "system", "content": system_prompt}
]

def chat(self, mensaje_usuario):
    """Envía un mensaje y obtiene la respuesta manteniendo contexto."""
    # Añadir mensaje del usuario
    self.mensajes.append({"role": "user", "content": mensaje_usuario})

    # Gestionar ventana de contexto
    self._gestionar_contexto()

    # Llamar a la API
    response = client.chat.completions.create(
        model=self.modelo,
        messages=self.mensajes
    )

    # Extraer y almacenar la respuesta
    respuesta = response.choices[0].message.content
    self.mensajes.append({"role": "assistant", "content": respuesta})

    return respuesta

def _gestionar_contexto(self):
    """Trunca el historial si excede el máximo de mensajes."""
    if len(self.mensajes) > self.max_mensajes:
        # Mantener el system prompt + los últimos N mensajes
        self.mensajes = [self.mensajes[0]] + self.mensajes[-(self.max_mensajes - 1):]

# Uso
bot = Chatbot(
    system_prompt="Eres un asistente de soporte técnico para una empresa de software.",
    max_mensajes=20
)

print(bot.chat("Hola, tengo un problema con la instalación"))
print(bot.chat("Me da un error de permisos"))
print(bot.chat("¿Qué sistema operativo me recomendaste antes?")) # Tiene contexto

```

FLUJO DE MEMORIA DEL CHATBOT:

```

Turno 1: [system] + [user: "Hola, tengo un problema..."]
        → [assistant: "¡Hola! Lamento que tengas..."]

Turno 2: [system] + [user: "Hola..."] + [assistant: "¡Hola!..."]
        + [user: "Me da un error de permisos"]
        → [assistant: "Un error de permisos puede..."]

Turno 3: [system] + [todo el historial anterior]
        + [user: "¿Qué sistema operativo...?"]
        → El modelo RECUERDA el contexto previo

```

4.2 Análisis de Sentimiento Automatizado

Procesamiento por lotes de textos para clasificar su sentimiento de forma automática.

Código completo: [sentimiento.py](#)

Patrón de Procesamiento por Lotes

```

from openai import OpenAI
import json

client = OpenAI()

def analizar_sentimiento_lote(textos, modelo="gpt-4o-mini"):
    """
    Analiza el sentimiento de una lista de textos.

    Args:
        textos: Lista de strings a analizar
        modelo: Modelo a utilizar
    """

```

```

Returns:
    Lista de diccionarios con sentimiento y confianza
"""
resultados = []

for i, texto in enumerate(textos):
    response = client.chat.completions.create(
        model=modelo,
        messages=[
            {
                "role": "system",
                "content": "Analiza el sentimiento del texto. "
                "Responde SOLO con JSON válido: "
                '{"sentimiento": "positivo|negativo|neutral", '
                '"confianza": 0.0-1.0, '
                '"razón": "breve explicación"}'
            },
            {
                "role": "user",
                "content": texto
            }
        ],
        temperature=0.0 # Determinista para clasificación
    )

    try:
        resultado = json.loads(response.choices[0].message.content)
        resultado["texto_original"] = texto[:100]
        resultados.append(resultado)
    except json.JSONDecodeError:
        resultados.append({
            "texto_original": texto[:100],
            "sentimiento": "error",
            "confianza": 0.0,
            "razón": "Error al parsear respuesta"
        })

    # Progreso
    print(f"Procesado {i+1}/{len(textos)}")

return resultados

# Ejemplo de uso
reseñas = [
    "El producto es excelente, superó mis expectativas",
    "Pésimo servicio, llevo 3 días esperando",
    "El envío llegó a tiempo, producto correcto",
    "No funciona como esperaba, muy decepcionado",
]

resultados = analizar_sentimiento_lote(reseñas)
for r in resultados:
    print(f"{r['sentimiento']}>10} ({r['confianza']:.0%}) - {r['texto_original']}")

```

4.3 Extracción Estructurada de Información (JSON)

Convertir texto no estructurado en datos estructurados que pueden procesarse programáticamente.

Código completo: [estructurada.py](#)

Patrón de Extracción

```

from openai import OpenAI
import json

client = OpenAI()

def extraer_informacion(texto, esquema_json):
    """
    Extrae información estructurada de texto libre.

    Args:
        texto: Texto del que extraer información
        esquema_json: Ejemplo del formato JSON esperado

    Returns:

```

```

    Diccionario con la información extraída
    """
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": f"Extrae la información del texto proporcionado. "
                    f"Responde SÓLO con JSON válido siguiendo este "
                    f"esquema:\n{json.dumps(esquema_json, indent=2)}\n\n"
                    f"Si algún campo no se encuentra en el texto, "
                    f"usa null."
            },
            {
                "role": "user",
                "content": texto
            }
        ],
        temperature=0.0,
        response_format={"type": "json_object"}
    )

    return json.loads(response.choices[0].message.content)

```

```

# Ejemplo: Extraer datos de un CV en texto libre
texto_cv = """
Me llamo María García López y tengo 28 años. Soy ingeniera informática
graduada de la Universidad Politécnica de Madrid en 2019. Actualmente
trabajo como desarrolladora senior en Acme Corp desde 2021. Domino
Python, JavaScript y Go. Mi email es maria.garcia@email.com y vivo
en Madrid. Hablo español nativo e inglés avanzado (C1).
"""

```

```

esquema = {
    "nombre_completo": "string",
    "edad": "number",
    "email": "string",
    "ciudad": "string",
    "educacion": {
        "titulo": "string",
        "universidad": "string",
        "año_graduacion": "number"
    },
    "experiencia_actual": {
        "puesto": "string",
        "empresa": "string",
        "desde": "number"
    },
    "habilidades_tecnicas": ["string"],
    "idiomas": [{"idioma": "string", "nivel": "string"}]
}

```

```

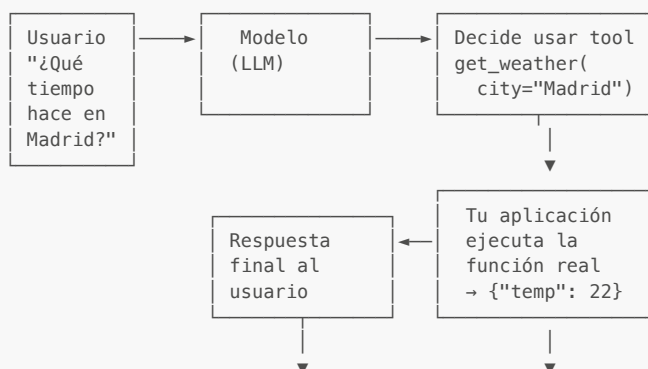
resultado = extraer_informacion(texto_cv, esquema)
print(json.dumps(resultado, indent=2, ensure_ascii=False))

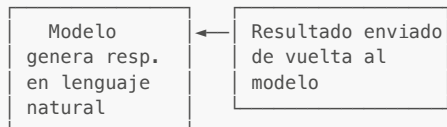
```

4.4 Function Calling / Tool Use

Function Calling (o Tool Use) permite que el modelo decida cuándo necesita invocar una herramienta externa para completar una tarea. El modelo no ejecuta la función: genera los argumentos y la aplicación la ejecuta.

FLUJO DE FUNCTION CALLING:





Ejemplo con OpenAI

```

import json
from openai import OpenAI

client = OpenAI()

# 1. Definir las herramientas disponibles
tools = [
    {
        "type": "function",
        "function": {
            "name": "obtener_clima",
            "description": "Obtiene el clima actual de una ciudad",
            "parameters": {
                "type": "object",
                "properties": {
                    "ciudad": {
                        "type": "string",
                        "description": "Nombre de la ciudad"
                    },
                    "unidad": {
                        "type": "string",
                        "enum": ["celsius", "fahrenheit"],
                        "description": "Unidad de temperatura"
                    }
                },
                "required": ["ciudad"]
            }
        }
    }
]

# 2. Primera llamada: el modelo decide si usar herramientas
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "¿Qué tiempo hace en Madrid?"}],
    tools=tools,
    tool_choice="auto"
)

# 3. Si el modelo quiere usar una herramienta
if response.choices[0].message.tool_calls:
    tool_call = response.choices[0].message.tool_calls[0]
    args = json.loads(tool_call.function.arguments)

# 4. Tu aplicación ejecuta la función real
resultado_clima = {"temperatura": 22, "condición": "soleado"}

# 5. Enviar resultado al modelo para respuesta final
response_final = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "user", "content": "¿Qué tiempo hace en Madrid?"},
        response.choices[0].message,
        {
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": json.dumps(resultado_clima)
        }
    ]
)
print(response_final.choices[0].message.content)
  
```

Conexión con la Unidad 4: Function Calling es la base fundamental de los **Agentes**, que estudiaremos en profundidad en la Unidad 4.

4.5 Generación de Embeddings

Los **embeddings** son representaciones vectoriales de texto. Convierten texto en vectores numéricos de alta dimensión (por ejemplo, 1536 dimensiones) que capturan el significado semántico.

CONCEPTO DE EMBEDDINGS:

"El gato duerme" → [0.023, -0.041, 0.087, ..., 0.012] (1536 dims)
"El felino reposa" → [0.025, -0.039, 0.091, ..., 0.010] (1536 dims)
"Python es genial" → [0.891, 0.234, -0.567, ..., 0.445] (1536 dims)

Los vectores de "gato duerme" y "felino reposa" son MUY similares porque tienen significado semántico cercano (similitud coseno ≈ 0.95).

Los vectores de "gato duerme" y "Python es genial" son MUY diferentes (similitud coseno ≈ 0.15).

Código completo: [embeddings/openai.py](#)

Generación y Comparación de Embeddings

```
from openai import OpenAI
import numpy as np

client = OpenAI()

def obtener_embedding(texto, modelo="text-embedding-3-small"):
    """Obtiene el embedding de un texto."""
    response = client.embeddings.create(
        input=texto,
        model=modelo
    )
    return response.data[0].embedding

def similitud_coseno(vec_a, vec_b):
    """Calcula la similitud coseno entre dos vectores."""
    a = np.array(vec_a)
    b = np.array(vec_b)
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Ejemplo de uso
textos = [
    "El gato duerme en el sofá",
    "El felino descansa en el sillón",
    "Python es un lenguaje de programación",
]

embeddings = [obtener_embedding(t) for t in textos]

# Comparar similitudes
for i in range(len(textos)):
    for j in range(i + 1, len(textos)):
        sim = similitud_coseno(embeddings[i], embeddings[j])
        print(f"Similitud entre:")
        print(f"  '{textos[i]}'")
        print(f"  '{textos[j]}'")
        print(f"  → {sim:.4f}\n")
```

Conexión con la Unidad 5: Los embeddings son la base fundamental de **RAG (Retrieval-Augmented Generation)**, que estudiaremos en la Unidad 5. Se usan para buscar documentos semánticamente relevantes en bases de datos vectoriales.

4.6 Patrones de Diseño y Buenas Prácticas

Patrón	Descripción	Beneficio
Cliente unificado	Una sola instancia del cliente en toda la aplicación	Reutilización de conexiones
Configuración externa	Parámetros en archivos de config, no hardcoded	Flexibilidad sin cambiar código
Logging estructurado	Registrar cada llamada con tokens, latencia, modelo	Monitorización y debugging
Retry con backoff	Reintentos automáticos con espera exponencial	Resiliencia ante errores transitorios
Validación de schemas	Verificar que la respuesta cumple el formato esperado	Robustez en procesamiento
Variables de entorno	API Keys y secretos fuera del código	Seguridad
Timeouts	Límite de tiempo para cada llamada	Evitar bloqueos indefinidos
Límites de tokens	Controlar <code>max_tokens</code> en cada llamada	Control de costos

Código de referencia para patrones: Consultar los archivos del repositorio [ml2_code](#) para implementaciones completas de cada patrón.

Bloque 5: Introducción a LangChain (40 minutos)

5.1 ¿Por qué un Framework de Orquestación?

Cuando trabajamos directamente con las APIs nativas de cada proveedor, nos enfrentamos a varios problemas recurrentes.

Problemas con APIs Nativas

PROBLEMAS CON APIs NATIVAS

1. Código específico por proveedor

→ Si cambias de OpenAI a Claude, reescribes todo

2. Gestión manual de memoria

→ Tú controlas el historial, truncado, resumen

3. Prompts no reutilizables

→ Strings hardcoded, sin templates parametrizados

4. Composición manual

→ Encadenar llamadas requiere código ad-hoc

5. Sin abstracciones comunes

→ Cada proyecto reinventa las mismas soluciones

Componentes de LangChain

LangChain proporciona abstracciones modulares para construir aplicaciones con LLMs.

Componente	Descripción	Ejemplo de Uso
Model I/O	Interfaz unificada para múltiples proveedores	Cambiar de OpenAI a Claude con una línea
Prompts	Templates parametrizados y reutilizables	Prompt con variables {idioma}, {tema}
Chains	Composición de operaciones secuenciales	Prompt → LLM → Parser encadenados
Memory	Gestión automática del historial	Buffer, resumen, ventana deslizante
Agents	Modelos que deciden qué herramientas usar	Agente que busca en web y calcula
Retrieval	Integración con bases de datos vectoriales	RAG con documentos propios

5.2 Cuándo Usar LangChain vs API Nativa

Escenario	Recomendación	Razón
Llamada simple, un solo proveedor	API Nativa	Menos dependencias, más control
Prototipado rápido	LangChain	Abstracciones aceleran desarrollo
Necesidad de cambiar proveedores	LangChain	Interfaz unificada
Aplicación compleja con memoria	LangChain	Gestión automática del contexto
Control total de la petición HTTP	API Nativa	Sin capas intermedias
Composición de múltiples pasos	LangChain	Operador pipe simplifica cadenas
Producción con requisitos estrictos	API Nativa	Menos puntos de fallo
Integración con vector stores / RAG	LangChain	Integraciones preconfiguradas

5.3 Chat Models Unificados

Código completo: [LangChain/componente1.py](#)

Instalación

```
pip install langchain langchain-openai langchain-anthropic langchain-google-genai
```

Misma Interfaz para Diferentes Proveedores

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain_google_genai import ChatGoogleGenerativeAI

# Crear modelos con la misma interfaz
modelo_openai = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)
modelo_claude = ChatAnthropic(model="claude-3-5-sonnet-20241022", temperature=0.7)
modelo_gemini = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0.7)

# También se puede usar OpenRouter con LangChain (gratis)
modelo_openrouter = ChatOpenAI(
    model="google/gemini-2.0-flash-exp:free",
    temperature=0.7,
    openai_api_key=os.getenv("OPENROUTER_API_KEY"),
    openai_api_base="https://openrouter.ai/api/v1",
)

# La misma llamada funciona con cualquiera
from langchain_core.messages import HumanMessage, SystemMessage

mensajes = [
    SystemMessage(content="Eres un asistente útil."),
    HumanMessage(content="¿Qué es Python?")
]

# Cambiar de proveedor es cambiar una variable
modelo_activo = modelo_openai # o modelo_claude o modelo_gemini
respuesta = modelo_activo.invoke(mensajes)
print(respuesta.content)
```

5.4 Prompt Templates

Los Prompt Templates permiten crear prompts parametrizados y reutilizables.

Código completo: [LangChain/componente2.py](#)

```
from langchain_core.prompts import ChatPromptTemplate

# Definir un template reutilizable
template = ChatPromptTemplate.from_messages([
    ("system", "Eres un traductor experto. Traduces de {idioma_origen} a {idioma_destino}."),
    ("human", "Traduce el siguiente texto:\n\n{texto}")
])

# Usar el template con diferentes parámetros
prompt_1 = template.invoke({
    "idioma_origen": "español",
    "idioma_destino": "inglés",
    "texto": "La inteligencia artificial está transformando el mundo."
})

prompt_2 = template.invoke({
    "idioma_origen": "español",
    "idioma_destino": "francés",
    "texto": "Buenos días, ¿cómo estás?"
})

# Enviar al modelo
from langchain_openai import ChatOpenAI
modelo = ChatOpenAI(model="gpt-4o-mini")

respuesta_1 = modelo.invoke(prompt_1)
respuesta_2 = modelo.invoke(prompt_2)
print(respuesta_1.content)
print(respuesta_2.content)
```

5.5 Chains (Composición de Operaciones)

Las Chains permiten encadenar operaciones usando el operador pipe |.

Código completo: [LangChain/componente3.py](#), [componente3_1.py](#)

Chain Básica con Operador Pipe

```

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Definir componentes
prompt = ChatPromptTemplate.from_messages([
    ("system", "Eres un experto en {tema}. Responde de forma concisa."),
    ("human", "{pregunta}")
])

modelo = ChatOpenAI(model="gpt-4o-mini")
parser = StrOutputParser()

# Componer la cadena con el operador pipe
cadena = prompt | modelo | parser

# Ejecutar
resultado = cadena.invoke({
    "tema": "Python",
    "pregunta": "¿Cuáles son las ventajas de usar type hints?"
})
print(resultado)

```

Cadenas Secuenciales

```

from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

modelo = ChatOpenAI(model="gpt-4o-mini")
parser = StrOutputParser()

# Paso 1: Generar un resumen
prompt_resumen = ChatPromptTemplate.from_messages([
    ("human", "Resume el siguiente texto en 2 oraciones:\n\n{texto}")
])

# Paso 2: Traducir el resumen
prompt_traducccion = ChatPromptTemplate.from_messages([
    ("human", "Traduce al inglés:\n\n{resumen}")
])

# Cadena secuencial
cadena_resumen = prompt_resumen | modelo | parser
cadena_traducccion = prompt_traducccion | modelo | parser

# Ejecutar secuencialmente
texto_largo = "La inteligencia artificial generativa ha experimentado..."
resumen = cadena_resumen.invoke({"texto": texto_largo})
traducccion = cadena_traducccion.invoke({"resumen": resumen})
print(traducccion)

```

5.6 Memory

La Memory de LangChain gestiona automáticamente el historial de conversación.

Código completo: [LangChain/componente4.py](#)

```

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

# Almacén de historiales por sesión
store = {}

def obtener_historial(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

# Crear la cadena con placeholder para historial
prompt = ChatPromptTemplate.from_messages([
    ("system", "Eres un tutor de Python amigable."),
    MessagesPlaceholder(variable_name="history"),

```

```
    ("human", "{input}")
  })

  modelo = ChatOpenAI(model="gpt-4o-mini")
  cadena = prompt | modelo

  # Envolver con gestión de historial
  cadena_con_memoria = RunnableWithMessageHistory(
    cadena,
    obtener_historial,
    input_messages_key="input",
    history_messages_key="history"
  )

  # Usar con sesiones
  config = {"configurable": {"session_id": "usuario_1"}}

  r1 = cadena_con_memoria.invoke(
    {"input": "¿Qué es una list comprehension?"},
    config=config
  )
  print(r1.content)

  r2 = cadena_con_memoria.invoke(
    {"input": "¿Puedes darme un ejemplo más complejo?"},
    config=config # Misma sesión → recuerda el contexto
  )
  print(r2.content)
```

5.7 Comparativa Final: API Nativa vs LangChain

Aspecto	API Nativa	LangChain
Curva de aprendizaje	Baja (solo SDK del proveedor)	Media (conceptos propios del framework)
Control	Total	Abstraído (menor control fino)
Código necesario	Más líneas para funcionalidad compleja	Menos líneas gracias a abstracciones
Cambiar proveedor	Reescribir código	Cambiar una línea
Memoria	Implementación manual	Módulos preconfigurados
Prompts	Strings manuales	Templates parametrizados
Composición	Funciones encadenadas manualmente	Operador pipe
Dependencias	Mínimas (solo SDK)	Múltiples paquetes
Debugging	Directo y transparente	Más capas, puede ser opaco
Documentación	Excelente (oficial del proveedor)	Buena pero cambiante

5.8 Conexión con Unidades Siguientes

LangChain no es solo un wrapper de APIs. Es la base para los temas avanzados que veremos en las próximas unidades.

ROADMAP DE LangChain EN EL CURSO:

Unidad 3 (actual):

Model I/O + Prompts + Chains + Memory

- └ Bases del framework

Unidad 4 (Agentes):

Agents + Tools

- └ create_react_agent, herramientas personalizadas
- └ El modelo DECIDE qué herramientas usar

Unidad 5 (RAG):

Retrieval + Vector Stores + Embeddings

- └ Bases de datos vectoriales (Chroma, FAISS)
- └ Documentos propios como contexto

Unidad 6 (MCP):

Model Context Protocol

- └ Herramientas desacopladas del modelo
- └ Protocolo estándar para conectar tools

Resumen de la Sesión

Conceptos Clave

1. **Arquitectura de APIs de LLMs:** Peticiones REST, autenticación, estructura de costos y rate limiting
2. **Cuatro formas de acceso:** OpenAI, Google Gemini, Anthropic Claude (APIs directas) y OpenRouter (gateway unificado con modelos gratuitos)
3. **Patrones prácticos:** Chatbot con memoria, análisis de sentimiento por lotes, extracción estructurada (JSON), Function Calling y embeddings
4. **Producción:** Manejo robusto de errores, streaming, gestión segura de API Keys y optimización de costos
5. **LangChain:** Framework de orquestación que unifica proveedores, ofrece templates de prompts, chains composables y gestión de memoria

Qué Deberías Saber Hacer

Habilidad	Nivel Esperado
Configurar API Keys de forma segura	Implementar con variables de entorno o .env
Hacer llamadas a OpenAI, Gemini y Claude	Código funcional con cada SDK
Manejar errores y rate limits	Retry con exponential backoff
Implementar streaming	Respuestas progresivas token a token
Construir un chatbot con memoria	Gestión del historial y contexto
Extraer datos estructurados	Texto libre → JSON con schema definido
Usar LangChain básico	Chat models, templates, chains simples

Conexión con la Práctica

La práctica evaluable de esta sesión está disponible en [practica.md](#).

Proyecto: Generador de Titulares con Miniature GPT - Aplicarás los conceptos de acceso programático a LLMs para construir una aplicación que genere titulares de noticias, integrando las APIs estudiadas en esta sesión con patrones de diseño de producción.

Actividades

Ejercicios de esta Sesión

Completa los ejercicios prácticos disponibles en [ejercicios.md](#):

1. **Configuración de APIs** - Configurar credenciales y hacer primera llamada a cada proveedor
2. **Comparativa de proveedores** - Mismo prompt en OpenAI, Gemini y Claude; comparar respuestas
3. **Chatbot con memoria** - Implementar un chatbot que mantenga contexto de conversación
4. **Análisis de sentimiento** - Procesar un lote de textos y clasificar sentimientos
5. **Extracción estructurada** - Convertir texto libre a JSON con schema definido
6. **Cadenas con LangChain** - Construir una chain multi-paso con templates y parsers

Referencias

- OpenAI. (2024). API Reference - Chat Completions. <https://platform.openai.com/docs/api-reference/chat>
- Anthropic. (2024). API Reference - Messages. <https://docs.anthropic.com/en/api/messages>
- Google. (2024). Gemini API Documentation. <https://ai.google.dev/docs>
- LangChain. (2024). Documentation. <https://python.langchain.com/docs/>
- OpenAI. (2024). Embeddings Guide. <https://platform.openai.com/docs/guides/embeddings>
- OpenAI. (2024). Function Calling Guide. <https://platform.openai.com/docs/guides/function-calling>
- OpenRouter. (2025). Documentation & Free Models. <https://openrouter.ai/docs>
- Repositorio del curso: https://github.com/rpmaya/ml2_code